

# Trabalho Prático

## Fundamentos de Projeto e Análise de Algoritmos

Gustavo Azi Prehl Gama  
Código de Pessoa: 754737

11 de junho de 2026

## Introdução

Este documento reúne as 10 questões do trabalho prático de FPA, resolvidas em LeetCode e Codeforces. Para cada questão são apresentados: (i) o código original submetido; (ii) a cópia integral da tela com a mensagem de aceite da plataforma; e (iii) uma justificativa de que o código resolve o problema, fundamentada na teoria de projeto e análise de algoritmos vista em sala (CLRS09 – Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*, 3ª ed.).

Em todas as questões, as análises de complexidade de tempo e espaço seguem a notação assintótica ( $O$ ,  $\Theta$ ,  $\Omega$ ) apresentada em *Notação Assintótica* [CLRS09, capítulo 3], complementando as referências específicas de cada estratégia indicadas em cada justificativa.

## Sumário

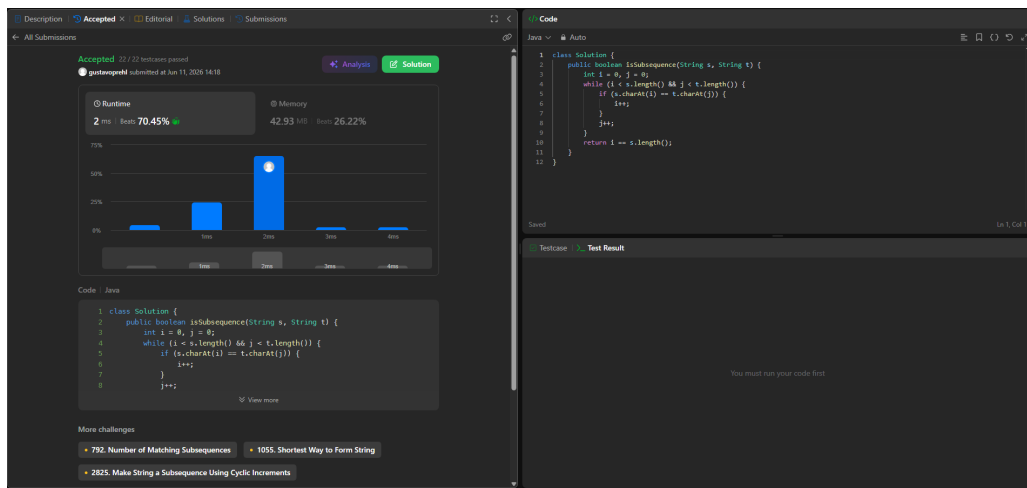
|    |                                                    |    |
|----|----------------------------------------------------|----|
| 1  | 392 - Is Subsequence (LeetCode)                    | 2  |
| 2  | 198 - House Robber (LeetCode)                      | 4  |
| 3  | 1042 - Flower Planting With No Adjacent (LeetCode) | 6  |
| 4  | 3286 - Find a Safe Walk Through a Grid (LeetCode)  | 8  |
| 5  | 52 - N-Queens II (LeetCode)                        | 10 |
| 6  | 2360 - Longest Cycle in a Graph (LeetCode)         | 12 |
| 7  | 516 - Longest Palindromic Subsequence (LeetCode)   | 14 |
| 8  | 1971 - Find if Path Exists in Graph (LeetCode)     | 16 |
| 9  | 654 - Maximum Binary Tree (LeetCode)               | 18 |
| 10 | E - The Lakes (Codeforces)                         | 20 |

# 1 392 - Is Subsequence (LeetCode)

## Código

```
1 class Solution {
2     public boolean isSubsequence(String s, String t) {
3         int i = 0, j = 0;
4         while (i < s.length() && j < t.length()) {
5             if (s.charAt(i) == t.charAt(j)) {
6                 i++;
7             }
8             j++;
9         }
10        return i == s.length();
11    }
12 }
```

## Comprovante de aceite



## Justificativa

**Estratégia:** dois ponteiros (técnica gulosa).

A solução percorre  $s$  (ponteiro  $i$ ) e  $t$  (ponteiro  $j$ ) simultaneamente. Sempre que  $s[i] == t[j]$ , o caractere de  $s$  é “consumido” ( $i++$ ); o ponteiro  $j$  avança a cada iteração, independentemente de ter havido correspondência.  $s$  é subsequência de  $t$  se, ao final,  $i$  percorreu todos os caracteres de  $s$ .

**Por que essa escolha gulosa é ótima (correção):** dado um caractere  $s[i]$  e a primeira ocorrência possível dele em  $t$  a partir da posição  $j$ , casar  $s[i]$  com essa primeira ocorrência nunca é pior do que casá-lo com uma ocorrência posterior. Isso porque qualquer correspondência válida para o restante de  $s$  que use uma posição  $j' > j$  também é válida (ou mais restrita) se usarmos  $j$ , já que sobra mais “espaço” em  $t$  à direita. Portanto, a escolha gulosa de avançar assim que há correspondência preserva a existência de uma solução completa, sem necessidade de backtracking. Esse argumento de troca (*exchange argument*) é a técnica padrão de prova de corretude de algoritmos gulosos.

**Complexidade:**

- Tempo:  $O(|t|)$ , pois cada ponteiro avança no máximo  $|t|$  vezes.

- Espaço:  $O(1)$ , apenas dois índices inteiros.

**Follow-up** ( $k \geq 10^9$  consultas sobre o mesmo  $\mathbf{t}$ ): pré-processar  $\mathbf{t}$  uma única vez, construindo, para cada caractere  $\mathbf{c}$ , a lista ordenada  $\mathbf{pos}[\mathbf{c}]$  com os índices onde  $\mathbf{c}$  ocorre em  $\mathbf{t}$  -  $O(|t|)$ . Para cada nova string  $s_i$ , percorrer seus caracteres e, para cada um, fazer busca binária em  $\mathbf{pos}[\mathbf{c}]$  pela menor posição estritamente maior que a posição atual em  $\mathbf{t}$ . Se não existir tal posição para algum caractere,  $s_i$  não é subsequência. Isso reduz o custo por consulta de  $O(|t|)$  para  $O(|s_i| \cdot \log |t|)$ , o que é essencial quando  $\mathbf{t}$  é fixo e há um número muito grande de consultas.

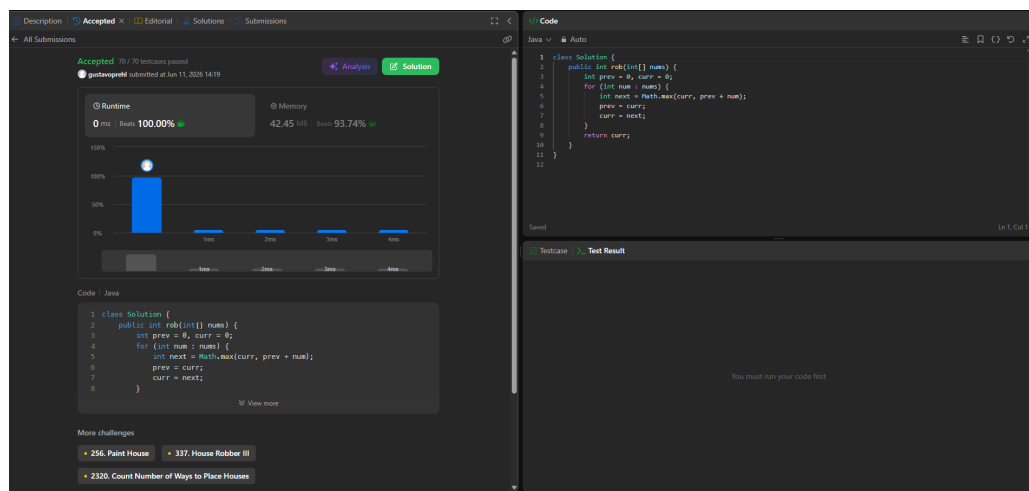
**Tópico do curso:** estratégia gulosa, com prova de corretude por argumento de troca - *Estratégia Gulosa e Divisão e Conquista* [CLRS09, caps. 4 e 16].

## 2 198 - House Robber (LeetCode)

### Código

```
1 class Solution {
2     public int rob(int[] nums) {
3         int prev = 0, curr = 0;
4         for (int num : nums) {
5             int next = Math.max(curr, prev + num);
6             prev = curr;
7             curr = next;
8         }
9         return curr;
10    }
11 }
```

### Comprovante de aceite



### Justificativa

**Estratégia:** programação dinâmica (*bottom-up*, com otimização de espaço).

**Subestrutura ótima:** seja  $dp[i]$  o valor máximo que se pode roubar considerando apenas as primeiras  $i$  casas. Para a casa  $i$ , há exatamente duas opções:

- **não roubar a casa  $i$ :** o melhor valor é  $dp[i - 1]$ ;
- **roubar a casa  $i$ :** como a casa  $i - 1$  não pode ter sido roubada, o melhor valor é  $dp[i - 2] + nums[i]$ .

Logo, a recorrência é:

$$dp[i] = \max(dp[i - 1], dp[i - 2] + nums[i])$$

A solução ótima global é construída a partir das soluções ótimas de subproblemas menores (prefixos do array), o que caracteriza a propriedade de **subestrutura ótima**.

**Subproblemas sobrepostos:**  $dp[i - 1]$  e  $dp[i - 2]$  são reutilizados repetidamente no cálculo de  $dp[i]$ ,  $dp[i + 1]$ , etc. Em vez de recalculá-los recursivamente (o que

levaria a complexidade exponencial), a abordagem *bottom-up* resolve cada subproblema uma única vez, em ordem crescente de  $i$ , armazenando o resultado.

**Otimização de espaço:** como  $dp[i]$  depende apenas dos dois valores anteriores, não é necessário manter o vetor  $dp$  inteiro - basta guardar **prev** ( $= dp[i-2]$ ) e **curr** ( $= dp[i-1]$ ), atualizando-os a cada iteração.

**Complexidade:**

- Tempo:  $O(n)$ , uma única passagem pelo array.
- Espaço:  $O(1)$ , apenas duas variáveis auxiliares.

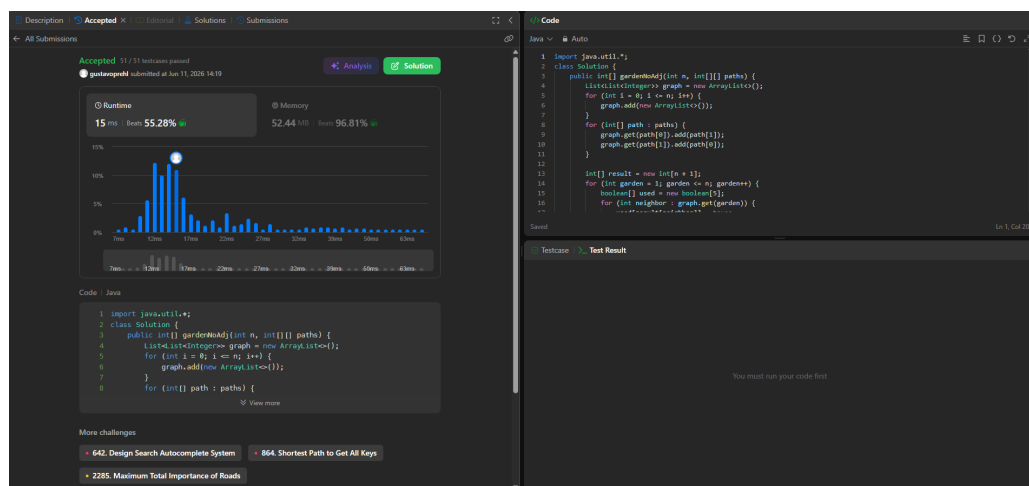
**Tópico do curso:** *Programação Dinâmica* [CLRS09, cap. 15] - identificação de subestrutura ótima e subproblemas sobrepostos, com formulação da recorrência *bottom-up* e otimização do espaço de armazenamento.

### 3 1042 - Flower Planting With No Adjacent (LeetCode)

#### Código

```
1 import java.util.*;
2
3 class Solution {
4     public int[] gardenNoAdj(int n, int[][] paths) {
5         List<List<Integer>> graph = new ArrayList<>();
6         for (int i = 0; i <= n; i++) {
7             graph.add(new ArrayList<>());
8         }
9         for (int[] path : paths) {
10             graph.get(path[0]).add(path[1]);
11             graph.get(path[1]).add(path[0]);
12         }
13
14         int[] result = new int[n + 1];
15         for (int garden = 1; garden <= n; garden++) {
16             boolean[] used = new boolean[5];
17             for (int neighbor : graph.get(garden)) {
18                 used[result[neighbor]] = true;
19             }
20             for (int color = 1; color <= 4; color++) {
21                 if (!used[color]) {
22                     result[garden] = color;
23                     break;
24                 }
25             }
26         }
27         return Arrays.copyOfRange(result, 1, n + 1);
28     }
29 }
```

#### Comprovante de aceite



#### Justificativa

**Estratégia:** coloração gulosa de grafos.

O problema é equivalente a colorir os vértices de um grafo (jardins = vértices, caminhos = arestas) de forma que vértices adjacentes recebam cores diferentes, usando até 4 cores (tipos de flores).

**Por que 4 cores sempre bastam (correção):** a restrição garante que cada vértice tem grau no máximo 3 (no máximo 3 caminhos incidentes). Pelo teorema da coloração gulosa, qualquer grafo pode ser colorido com no máximo  $\Delta + 1$  cores, onde  $\Delta$  é o grau máximo - basta processar os vértices em qualquer ordem e, para cada um, atribuir a menor cor que ainda não foi usada por nenhum de seus vizinhos já coloridos. Como  $\Delta \leq 3$ , sempre haverá pelo menos uma cor livre entre as 4 disponíveis, mesmo que todos os vizinhos já coloridos usem cores distintas entre si.

**Implementação:** constrói-se a lista de adjacência a partir de `paths` e, para cada jardim de 1 a  $n$ , marca-se em um vetor booleano `used[1..4]` quais cores já foram usadas pelos vizinhos processados anteriormente; a primeira cor livre é atribuída ao jardim atual.

**Complexidade:**

- Tempo:  $O(n + |\text{paths}|)$  - a construção do grafo é  $O(|\text{paths}|)$  e a coloração visita cada vértice e cada aresta um número constante de vezes (grau  $\leq 3$ ).
- Espaço:  $O(n + |\text{paths}|)$  para a lista de adjacência e o vetor de resultado.

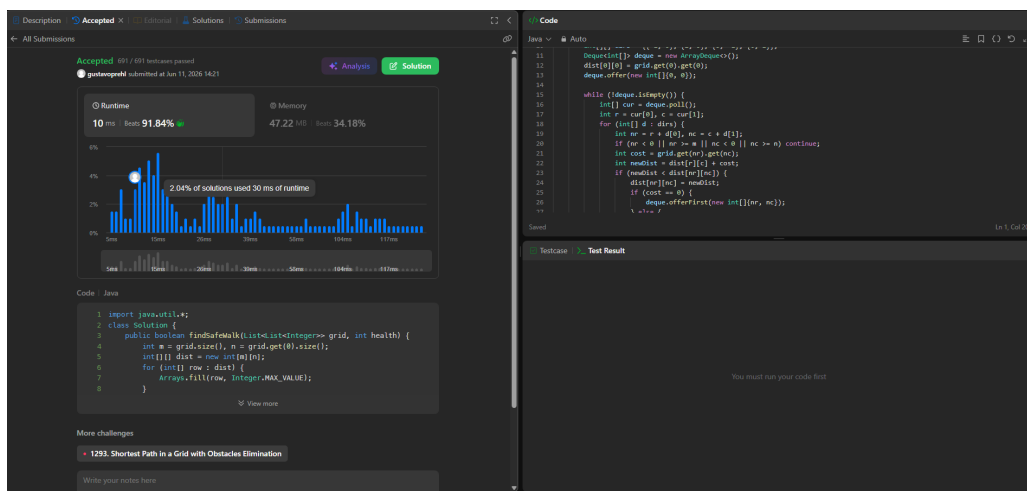
**Tópico do curso:** coloração gulosa baseada no teorema  $\Delta + 1$  - *Estratégia Gulosa e Divisão e Conquista* [CLRS09, caps. 4 e 16]. Como o algoritmo resolve o problema em tempo polinomial  $O(n + |\text{paths}|)$ , ele também ilustra a *Classe P* [CLRS09, cap. 34.1].

## 4 3286 - Find a Safe Walk Through a Grid (LeetCode)

### Código

```
1 import java.util.*;
2
3 class Solution {
4     public boolean findSafeWalk(List<List<Integer>> grid, int health) {
5         int m = grid.size(), n = grid.get(0).size();
6         int[][] dist = new int[m][n];
7         for (int[] row : dist) {
8             Arrays.fill(row, Integer.MAX_VALUE);
9         }
10
11         int[][] dirs = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
12         Deque<int[]> deque = new ArrayDeque<>();
13         dist[0][0] = grid.get(0).get(0);
14         deque.offer(new int[]{0, 0});
15
16         while (!deque.isEmpty()) {
17             int[] cur = deque.poll();
18             int r = cur[0], c = cur[1];
19             for (int[] d : dirs) {
20                 int nr = r + d[0], nc = c + d[1];
21                 if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
22                 int cost = grid.get(nr).get(nc);
23                 int newDist = dist[r][c] + cost;
24                 if (newDist < dist[nr][nc]) {
25                     dist[nr][nc] = newDist;
26                     if (cost == 0) {
27                         deque.offerFirst(new int[]{nr, nc});
28                     } else {
29                         deque.offerLast(new int[]{nr, nc});
30                     }
31                 }
32             }
33         }
34
35         return dist[m - 1][n - 1] <= health - 1;
36     }
37 }
```

# Comprovante de aceite



## Justificativa

**Estratégia:** caminho mínimo em grafo com pesos  $\{0, 1\}$  - algoritmo **0-1 BFS**, uma especialização do algoritmo de Dijkstra.

**Modelagem:** cada célula da grade é um vértice; há uma aresta entre células adjacentes (4 direções). O “custo” de entrar em uma célula  $\text{grid}[i][j] = 1$  é 1 (perde-se 1 ponto de vida) e o custo de uma célula  $\text{grid}[i][j] = 0$  é 0. Definimos  $\text{dist}[i][j]$  como o menor número total de células inseguras percorridas (incluindo a célula de partida) em qualquer caminho de  $(0, 0)$  até  $(i, j)$ . A resposta é **true** se e somente se  $\text{dist}[m-1][n-1] \leq \text{health} - 1$ , ou seja, se a vida restante ao chegar no destino for pelo menos 1.

**Por que o 0-1 BFS é correto (relação com Dijkstra):** o algoritmo de Dijkstra processa vértices em ordem não decrescente de distância, garantindo que, quando um vértice é retirado da estrutura, sua distância já é a definitiva (propriedade da escolha gulosa + relaxamento de arestas). Como aqui os pesos das arestas são apenas 0 ou 1, um deque substitui a fila de prioridade: ao relaxar uma aresta de peso 0, o vizinho é inserido no início do deque (mesma “camada” de distância); ao relaxar uma aresta de peso 1, o vizinho é inserido no final (próxima camada). Essa invariante mantém o deque sempre ordenado por distância, preservando a propriedade de Dijkstra sem necessidade de heap.

### Complexidade:

- Tempo:  $O(m \cdot n)$ , pois cada célula é inserida/relaxada um número constante de vezes (grau  $\leq 4$ ).
- Espaço:  $O(m \cdot n)$  para a matriz **dist** e o deque.

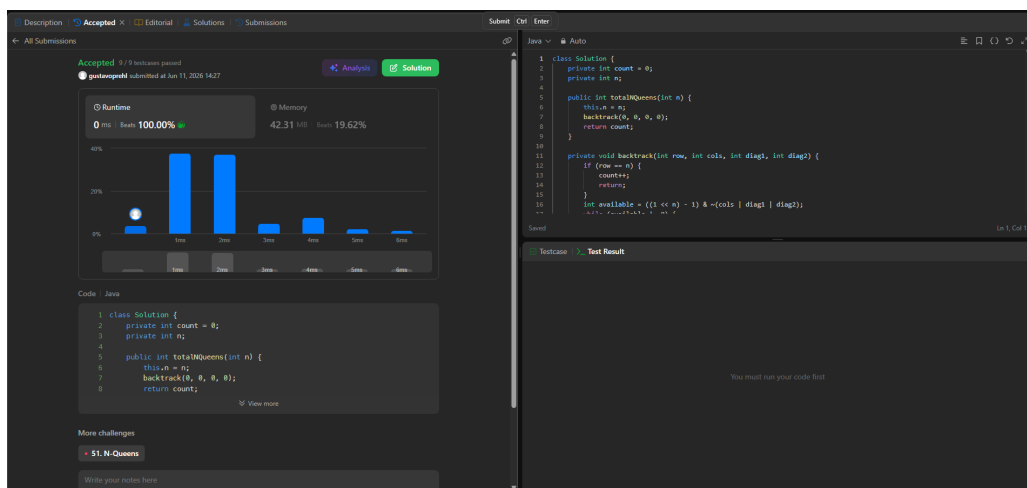
**Tópico do curso:** a escolha gulosa de sempre expandir primeiro o vértice de menor distância provisória (garantida pela ordenação do deque no 0-1 BFS) é a mesma propriedade de corretude do algoritmo de Dijkstra - *Estratégia Gulosa e Divisão e Conquista* [CLRS09, caps. 4 e 16].

## 5 52 - N-Queens II (LeetCode)

### Código

```
1 class Solution {
2     private int count = 0;
3     private int n;
4
5     public int totalNQueens(int n) {
6         this.n = n;
7         this.count = 0;
8         backtrack(0, 0, 0, 0);
9         return count;
10    }
11
12    private void backtrack(int row, int cols, int diag1, int diag2) {
13        if (row == n) {
14            count++;
15            return;
16        }
17        int available = ((1 << n) - 1) & ~(cols | diag1 | diag2);
18        while (available != 0) {
19            int pos = available & (-available);
20            available -= pos;
21            backtrack(row + 1, cols | pos, (diag1 | pos) << 1, (diag2 |
pos) >> 1);
22        }
23    }
24 }
```

### Comprovante de aceite



### Justificativa

**Estratégia:** *backtracking* (busca em profundidade no espaço de soluções) com poda via *bitmask*.

**Modelagem:** como duas rainhas nunca podem ficar na mesma linha, basta decidir, para cada linha de 0 a  $n - 1$ , em qual coluna colocar a rainha daquela linha. Isso reduz o

espaço de busca de  $2^{n^2}$  (todas as combinações de casas) para no máximo  $n^n$  (uma coluna por linha), e a poda reduz ainda mais na prática.

**Representação por bitmask:** três inteiros guardam, em cada posição de bit, se uma coluna (`cols`), uma diagonal principal (`diag1`) ou uma diagonal secundária (`diag2`) já está “sob ataque” pelas rainhas colocadas até a linha atual. A operação  $((1 \ll n) - 1) \& \sim(\text{cols} \mid \text{diag1} \mid \text{diag2})$  calcula em  $O(1)$  o conjunto de colunas livres para a linha atual.

**Poda (branch and bound):** ao avançar para a próxima linha, atualiza-se `cols`, `diag1` (deslocada à esquerda) e `diag2` (deslocada à direita) para refletir as novas casas atacadas. Se, em alguma linha, `available == 0`, o ramo é descartado imediatamente - não há necessidade de continuar explorando colocações que já violam as restrições do problema. Quando `row == n`, uma solução válida completa foi encontrada e o contador é incrementado.

**Complexidade:**

- Tempo: no pior caso  $O(N!)$ , mas a poda elimina a grande maioria dos ramos inválidos antes de chegar às últimas linhas (na prática, muito mais rápido que a enumeração ingênua).
- Espaço:  $O(N)$  para a profundidade da recursão (não há estruturas adicionais de tamanho  $O(N^2)$ ).

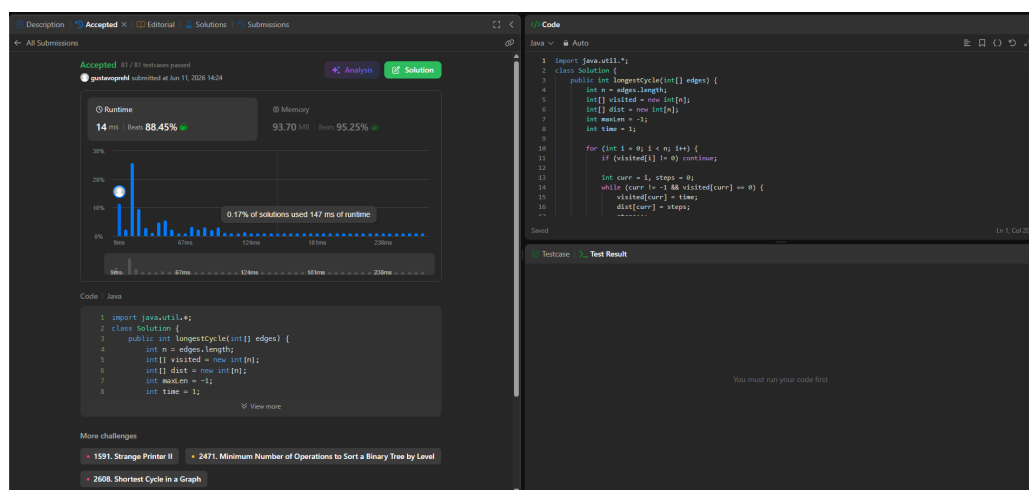
**Tópico do curso:** *Branch-and-Bound e Backtracking* - a busca explora o espaço de soluções por profundidade e poda ramos inviáveis assim que a inconsistência é detectada (`available == 0`), sem necessidade de explorá-los até o fim.

## 6 2360 - Longest Cycle in a Graph (LeetCode)

### Código

```
1 import java.util.*;
2
3 class Solution {
4     public int longestCycle(int[] edges) {
5         int n = edges.length;
6         int[] visited = new int[n];
7         int[] dist = new int[n];
8         int maxLen = -1;
9         int time = 1;
10
11         for (int i = 0; i < n; i++) {
12             if (visited[i] != 0) continue;
13
14             int curr = i, steps = 0;
15             while (curr != -1 && visited[curr] == 0) {
16                 visited[curr] = time;
17                 dist[curr] = steps;
18                 steps++;
19                 curr = edges[curr];
20             }
21             if (curr != -1 && visited[curr] == time) {
22                 maxLen = Math.max(maxLen, steps - dist[curr]);
23             }
24             time++;
25         }
26         return maxLen;
27     }
28 }
```

### Comprovante de aceite



### Justificativa

**Estratégia:** travessia iterativa de grafo funcional com marcação por *timestamp* (variação de busca em profundidade sem recursão/pilha extra).

**Estrutura do problema:** como cada nó tem grau de saída no máximo 1, o grafo é uma coleção de “componentes funcionais”: cada um é uma cadeia de nós que termina em  $-1$  (sem ciclo) ou que eventualmente entra em um único ciclo, em formato de  $\rho$  (rho). Isso significa que, a partir de qualquer nó, seguir os ponteiros `edges[i]` leva, no máximo, a um único ciclo.

**Algoritmo:** para cada nó  $i$  ainda não visitado, percorre-se a cadeia a partir dele, atribuindo a cada nó visitado nessa travessia um `timestamp` (identificador da travessia atual) e a sua distância (`dist`, número de passos desde  $i$ ). A travessia para quando:

- chega a  $-1$  (cadeia sem ciclo) - não há ciclo a contabilizar; ou
- chega a um nó já visitado:
  - se o `timestamp` desse nó for o **mesmo** da travessia atual, então o nó faz parte de um ciclo encontrado **agora**, e o tamanho do ciclo é `passos_atuais - dist[nó]` (a diferença entre quando o nó foi “reencontrado” e quando foi visitado pela primeira vez nesta mesma travessia);
  - se o `timestamp` for de uma travessia **anterior**, a cadeia atual apenas se conecta a um componente já processado, sem formar um novo ciclo.

**Por que cada nó é processado uma única vez (correção da complexidade):** uma vez que `visited[curr] != 0`, o nó nunca é revisitado por outra travessia (a condição `visited[curr] == 0` no laço `while` impede isso). Logo, cada nó é marcado e processado exatamente uma vez ao longo de todas as travessias.

**Complexidade:**

- Tempo:  $O(n)$ , pois cada nó é visitado uma única vez no total, somando todas as travessias.
- Espaço:  $O(n)$  para os vetores `visited` e `dist`.

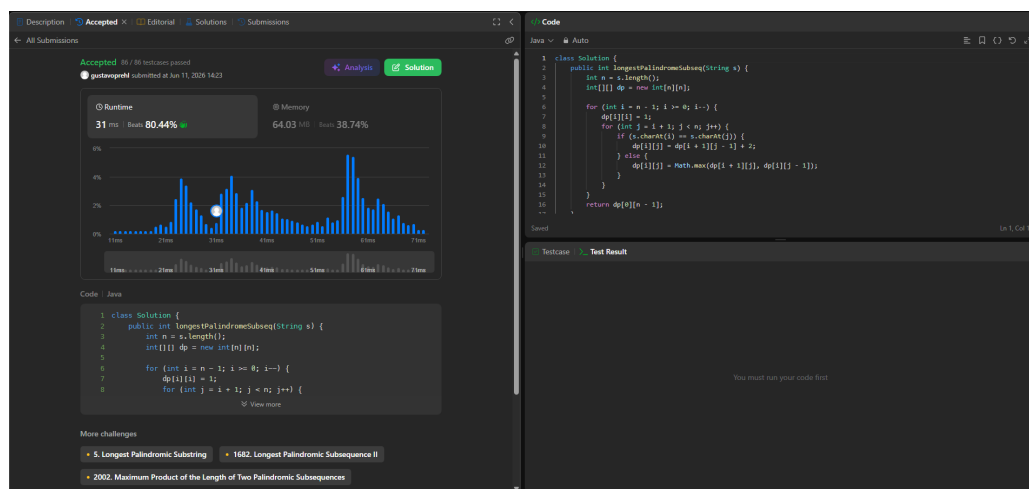
**Tópico do curso:** *Análise de Algoritmos Iterativos* [CLRS09, caps. 1 e 2.2] - o algoritmo é estruturado como laços iterativos (sem recursão), e o argumento de que cada nó é marcado e processado no máximo uma vez é o que garante o limite  $O(n)$  no total de iterações.

## 7 516 - Longest Palindromic Subsequence (LeetCode)

### Código

```
1 class Solution {
2     public int longestPalindromeSubseq(String s) {
3         int n = s.length();
4         int[][] dp = new int[n][n];
5
6         for (int i = n - 1; i >= 0; i--) {
7             dp[i][i] = 1;
8             for (int j = i + 1; j < n; j++) {
9                 if (s.charAt(i) == s.charAt(j)) {
10                     dp[i][j] = dp[i + 1][j - 1] + 2;
11                 } else {
12                     dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
13                 }
14             }
15         }
16         return dp[0][n - 1];
17     }
18 }
```

### Comprovante de aceite



### Justificativa

**Estratégia:** programação dinâmica em intervalos (*interval DP*).

**Definição do subproblema:**  $dp[i][j]$  representa o comprimento da maior subseqüência palíndroma contida em  $s[i..j]$  (substring de  $i$  a  $j$ , inclusive).

**Subestrutura ótima (recorrência):** compara-se os caracteres das extremidades do intervalo:

- Se  $s[i] == s[j]$ , ambos os caracteres podem fazer parte da subsequência palíndroma (um na ponta esquerda, outro na direita), e o problema se reduz a encontrar a maior subsequência palíndroma do intervalo interno  $s[i+1..j-1]$ , somando 2:  $dp[i][j] = dp[i+1][j-1] + 2$ .

- Se  $s[i] \neq s[j]$ , pelo menos um dos dois extremos não pode pertencer à subsequência ótima, então a resposta é o melhor entre ignorar a extremidade esquerda ou a direita:  $dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$ .

**Caso base:**  $dp[i][i] = 1$ , pois um único caractere é sempre um palíndromo de tamanho 1.

**Subproblemas sobrepostos e ordem de preenchimento:**  $dp[i][j]$  depende apenas de subintervalos menores ( $dp[i+1][j-1]$ ,  $dp[i+1][j]$ ,  $dp[i][j-1]$ ), todos com  $i$  maior ou  $j$  menor que o intervalo atual. Preenchendo  $i$  de  $n-1$  até 0 e, para cada  $i$ ,  $j$  de  $i+1$  até  $n-1$ , garante-se que toda dependência já foi calculada antes de ser usada (programação dinâmica *bottom-up*).

**Resposta final:**  $dp[0][n-1]$ , o intervalo que cobre a string inteira.

**Complexidade:**

- Tempo:  $O(n^2)$ , um valor para cada par  $(i, j)$  com  $i \leq j$ .
- Espaço:  $O(n^2)$  para a tabela  $dp$ .

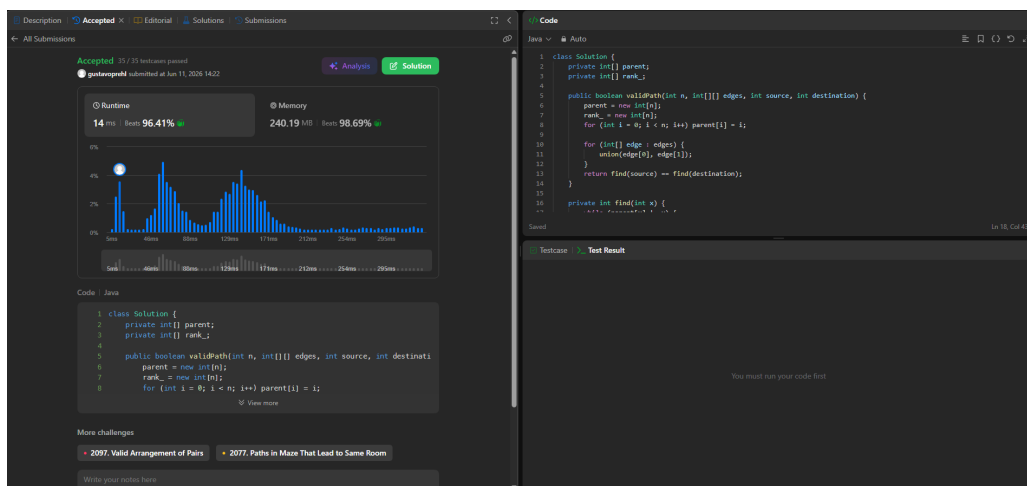
**Tópico do curso:** *Programação Dinâmica* [CLRS09, cap. 15] - formulação de  $dp[i][j]$  sobre intervalos da string, com recorrência baseada em subintervalos menores e preenchimento *bottom-up* respeitando as dependências entre subproblemas.

## 8 1971 - Find if Path Exists in Graph (LeetCode)

### Código

```
1 class Solution {
2     private int[] parent;
3     private int[] rank_;
4
5     public boolean validPath(int n, int[][] edges, int source, int
destination) {
6         parent = new int[n];
7         rank_ = new int[n];
8         for (int i = 0; i < n; i++) parent[i] = i;
9
10        for (int[] edge : edges) {
11            union(edge[0], edge[1]);
12        }
13        return find(source) == find(destination);
14    }
15
16    private int find(int x) {
17        while (parent[x] != x) {
18            parent[x] = parent[parent[x]]; // path halving
19            x = parent[x];
20        }
21        return x;
22    }
23
24    private void union(int a, int b) {
25        int ra = find(a), rb = find(b);
26        if (ra == rb) return;
27        if (rank_[ra] < rank_[rb]) {
28            int tmp = ra;
29            ra = rb;
30            rb = tmp;
31        }
32        parent[rb] = ra;
33        if (rank_[ra] == rank_[rb]) rank_[ra]++;
34    }
35 }
```

## Comprovante de aceite



## Justificativa

**Estratégia:** Union-Find (*Disjoint Set Union* - DSU) com compressão de caminho e união por *rank*.

**Modelagem:** existe um caminho entre *source* e *destination* se e somente se os dois vértices pertencem ao mesmo componente conexo do grafo. A estrutura DSU mantém uma partição dos vértices em conjuntos disjuntos, onde cada conjunto representa um componente conexo, identificado por um representante (raiz).

**Construção:** inicialmente cada vértice é seu próprio conjunto ( $\text{parent}[i] = i$ ). Para cada aresta  $(u, v)$  em *edges*, executa-se  $\text{union}(u, v)$ , unindo os conjuntos aos quais *u* e *v* pertencem. Ao final, basta verificar se  $\text{find}(\text{source}) == \text{find}(\text{destination})$ .

**Compressão de caminho (find):** ao buscar a raiz de um elemento, cada nó visitado é reconectado mais perto da raiz (aqui via *path halving*:  $\text{parent}[x] = \text{parent}[\text{parent}[x]]$ ), achatando a árvore e acelerando buscas futuras.

**União por rank (union):** ao unir dois conjuntos, a raiz da árvore de menor *rank* (altura estimada) é pendurada na raiz da árvore de maior *rank*, evitando que as árvores fiquem muito altas.

**Complexidade (teoria de DSU):** combinando compressão de caminho e união por *rank*, cada operação *find*/*union* tem custo amortizado  $O(\alpha(n))$ , onde  $\alpha$  é a inversa da função de Ackermann - na prática, uma constante muito pequena ( $\leq 4$  para qualquer *n* razoável).

- Tempo:  $O((n + |\text{edges}|) \cdot \alpha(n)) \approx O(n + |\text{edges}|)$ .
- Espaço:  $O(n)$  para os vetores *parent* e *rank\_*.

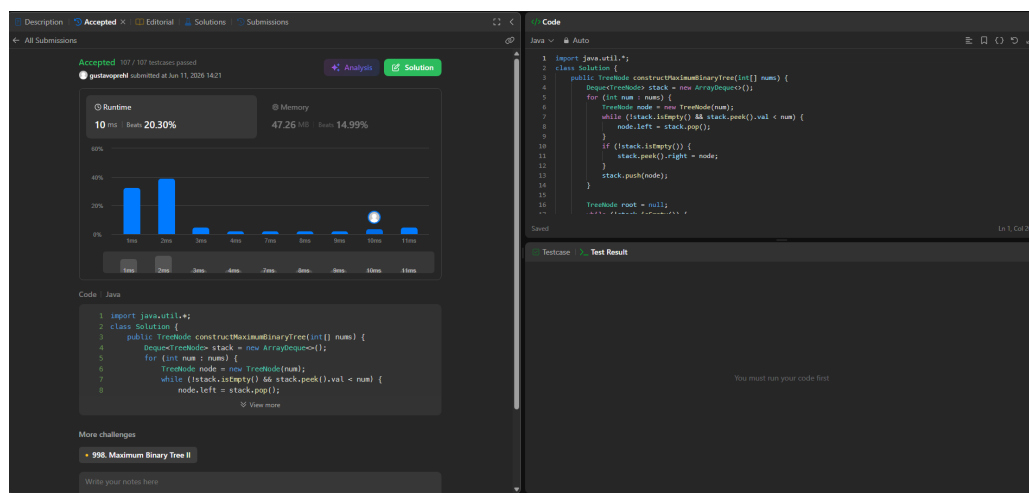
**Tópico do curso:** a construção do DSU é puramente iterativa (laços sobre os vértices e sobre *edges*), enquadrando-se em *Análise de Algoritmos Iterativos* [CLRS09, caps. 1 e 2.2]; como o tempo total é quase-linear,  $O((n + |\text{edges}|) \cdot \alpha(n))$ , o algoritmo também é um exemplo de problema na *Classe P* [CLRS09, cap. 34.1].

## 9 654 - Maximum Binary Tree (LeetCode)

### Código

```
1 import java.util.*;
2
3 class Solution {
4     public TreeNode constructMaximumBinaryTree(int[] nums) {
5         Deque<TreeNode> stack = new ArrayDeque<>();
6         for (int num : nums) {
7             TreeNode node = new TreeNode(num);
8             while (!stack.isEmpty() && stack.peek().val < num) {
9                 node.left = stack.pop();
10            }
11            if (!stack.isEmpty()) {
12                stack.peek().right = node;
13            }
14            stack.push(node);
15        }
16
17        TreeNode root = null;
18        while (!stack.isEmpty()) {
19            root = stack.pop();
20        }
21        return root;
22    }
23 }
```

### Comprovante de aceite



### Justificativa

**Estratégia:** pilha monotônica, construindo a árvore em uma única passagem (em vez da abordagem ingênua  $O(n^2)$  de buscar o máximo de cada subarray recursivamente).

**Invariante da pilha:** a pilha mantém, do fundo para o topo, os nós que ainda não encontraram, à sua direita, um valor maior que o seu - ou seja, candidatos a serem “dominados” por um número futuro. Os valores na pilha são estritamente decrescentes do

fundo para o topo, e cada nó é o filho direito do nó imediatamente abaixo dele (formando a “espinha direita” da árvore parcial construída até o momento).

**Relação com a definição recursiva do problema:** na árvore binária máxima, o pai de um elemento `nums[i]` é o **menor** entre o primeiro elemento maior à sua esquerda e o primeiro elemento maior à sua direita (o “dominador” mais próximo). Ao processar `nums` da esquerda para a direita:

- Se o número atual é **maior** que o topo da pilha, ele é o primeiro elemento maior à direita de todos os nós removidos - portanto, o número atual se torna ancestral deles. O último nó removido (o de maior valor entre os removidos, e portanto o mais próximo na hierarquia) vira o **filho esquerdo** do novo nó; os demais já estavam encadeados como filhos direitos uns dos outros, preservando a ordem relativa original.
- Se a pilha não ficou vazia após as remoções, o topo restante é o primeiro elemento maior à esquerda do número atual - logo, o número atual se torna **filho direito** desse nó.

**Análise amortizada (custo  $O(n)$ ):** cada elemento de `nums` é empilhado exatamente uma vez e desempilhado no máximo uma vez durante toda a execução. Pelo método agregado de análise amortizada, o número total de operações de pilha (*push* + *pop*) ao longo das  $n$  iterações é, no máximo,  $2n$ , mesmo que o laço `while` interno execute múltiplas vezes em iterações específicas.

**Complexidade:**

- Tempo:  $O(n)$  amortizado.
- Espaço:  $O(n)$  para a pilha (no pior caso, array estritamente crescente).

**Tópico do curso:** a definição do problema - o pai de cada elemento é o menor entre os “dominadores” à esquerda e à direita - corresponde a uma formulação recursiva de divisão e conquista (*Estratégia Gulosa e Divisão e Conquista* [CLRS09, caps. 4 e 16]); a implementação com pilha monotônica é a versão iterativa otimizada dessa recursão, cuja análise de custo amortizado  $O(n)$  se enquadra em *Análise de Algoritmos Iterativos* [CLRS09, caps. 1 e 2.2].

## 10 E - The Lakes (Codeforces)

### Código

```
1 import java.util.*;
2 import java.io.*;
3
4 public class Main {
5     public static void main(String[] args) throws IOException {
6         StreamTokenizer in = new StreamTokenizer(new BufferedReader(new
        InputStreamReader(System.in)));
7         StringBuilder sb = new StringBuilder();
8
9         in.nextToken();
10        int t = (int) in.nval;
11
12        while (t-- > 0) {
13            in.nextToken();
14            int n = (int) in.nval;
15            in.nextToken();
16            int m = (int) in.nval;
17
18            int[] a = new int[n * m];
19            for (int i = 0; i < n * m; i++) {
20                in.nextToken();
21                a[i] = (int) in.nval;
22            }
23
24            boolean[] visited = new boolean[n * m];
25            int[] queue = new int[n * m];
26            long best = 0;
27
28            for (int start = 0; start < n * m; start++) {
29                if (a[start] == 0 || visited[start]) continue;
30
31                int head = 0, tail = 0;
32                queue[tail++] = start;
33                visited[start] = true;
34                long volume = 0;
35
36                while (head < tail) {
37                    int cell = queue[head++];
38                    volume += a[cell];
39                    int r = cell / m, c = cell % m;
40
41                    if (r > 0) {
42                        int nb = cell - m;
43                        if (!visited[nb] && a[nb] > 0) {
44                            visited[nb] = true;
45                            queue[tail++] = nb;
46                        }
47                    }
48                    if (r < n - 1) {
49                        int nb = cell + m;
50                        if (!visited[nb] && a[nb] > 0) {
51                            visited[nb] = true;
52                            queue[tail++] = nb;
```

```

53     }
54 }
55 if (c > 0) {
56     int nb = cell - 1;
57     if (!visited[nb] && a[nb] > 0) {
58         visited[nb] = true;
59         queue[tail++] = nb;
60     }
61 }
62 if (c < m - 1) {
63     int nb = cell + 1;
64     if (!visited[nb] && a[nb] > 0) {
65         visited[nb] = true;
66         queue[tail++] = nb;
67     }
68 }
69 }
70
71 best = Math.max(best, volume);
72 }
73
74 sb.append(best).append('\n');
75 }
76
77 System.out.print(sb);
78 }
79 }

```

## Comprovante de aceite

CODEFORCES  
Sponsored by TON

gustavoprehl | Logout

HOME TOP CATALOG CONTESTS GYM PROBLEMS SET GROUPS RATING EDU API CALENDAR HELP

MAIN ACMSGURU | PROBLEMS SUBMIT STATUS STANDINGS CUSTOM TEST

Contest status

| #         | When                    | Who          | Problem           | Lang    | Verdict  | Time   | Memory  |
|-----------|-------------------------|--------------|-------------------|---------|----------|--------|---------|
| 378208498 | Jun/11/2026 11:10 UTC-3 | gustavoprehl | 1829F - The Lakes | Java 21 | Accepted | 406 ms | 8000 KB |

Sort by: ☐ Default order ☐ Submission time ☐ Judging Time ☐ Solution Size ☒ Execution Time

## Justificativa

**Estratégia:** componentes conexas em grade (BFS), com leitura rápida e fila explícita por questões de desempenho.

**Modelagem:** cada célula com  $a[i][j] > 0$  é um vértice de um grafo implícito, com arestas para os vizinhos ortogonais (cima, baixo, esquerda, direita) que também tenham valor  $> 0$ . Um “lago” é exatamente uma componente conexa desse grafo, e seu “volume” é a soma dos valores dos vértices da componente. O problema pede o maior volume entre todas as componentes - ou 0 se não existir nenhuma célula com valor  $> 0$ .

**Algoritmo:** percorre-se cada célula da grade; se ainda não foi visitada e tem valor  $> 0$ , executa-se uma BFS a partir dela, somando os valores de todas as células alcançadas (marcando-as como visitadas para não serem reprocessadas). O maior valor de soma encontrado entre todas as BFS é a resposta.

**Por que BFS iterativa com array em vez de DFS recursiva:** como  $n, m \leq 1000$  e a soma de  $n \cdot m$  pode chegar a  $10^6$ , uma única componente pode ter até  $10^6$  células. Uma

DFS recursiva nesse cenário atingiria profundidade de recursão da ordem de  $10^6$ , estourando a pilha de chamadas (`StackOverflowError`). A BFS com fila implementada como array de tamanho  $n \cdot m$  evita esse problema, processando cada célula em  $O(1)$  amortizado.

**Por que leitura rápida (`StreamTokenizer`):** com  $t \leq 10^4$  casos de teste e soma de  $n \cdot m \leq 10^6$ , o volume total de números a serem lidos pode chegar à ordem de milhões. `Scanner` é lento demais para essa escala (risco de *Time Limit Exceeded*); `StreamTokenizer` sobre um `BufferedReader` realiza a tokenização de forma eficiente, em tempo linear no tamanho da entrada.

**Complexidade (por caso de teste):**

- Tempo:  $O(n \cdot m)$  - cada célula é visitada e processada uma única vez.
- Espaço:  $O(n \cdot m)$  para os vetores `a`, `visited` e `queue`.

Como a soma de  $n \cdot m$  sobre todos os casos de teste é limitada a  $10^6$ , o tempo total do programa é  $O(10^6)$ , dentro do limite de 3 segundos.

**Tópico do curso:** o algoritmo é descrito inteiramente por laços iterativos (BFS com fila explícita e leitura via `StreamTokenizer`), conforme *Análise de Algoritmos Iterativos* [CLRS09, caps. 1 e 2.2]; sua complexidade  $O(n \cdot m)$  por caso de teste, linear no tamanho da entrada, é o tipo de cota que caracteriza problemas tratáveis na *Classe P* [CLRS09, cap. 34.1].